

Imperative programming

Jarmo Hurri

Quick dictionary

En	Fin	En	Fin
assignment	sijoittaminen	indentation	sisentäminen
branch	haarautua	integer	kokonaisluku
character	merkki	interpreter	tulkki
compiler	kääntäjä	loop	silmukka
concatenate	liittää	quotient	osamäärä
conditional	ehdollinen	ratio	suhde
constant	vakio	remainder	jakojäännös
division	jakolasku	repetition	toistaminen
equality	yhtäsuuruus	sequentially	peräkkäin
expression	lauseke	square	korottaa toiseen potenssiin
factorial	kertoma	statement	lause
flowchart	vuokaavio	string	merkkijono
formula	kaava	suite	lohko
grammar	kielioppi	variable	muuttuja

Overview

- what is programming
- imperative algorithms
 - state (variables)
 - statements that change state
 - control structures affecting flow
- representations for imperative algorithms
 - Python programs
 - flowcharts
- examples with math formulas (before logic, arrays and classes)
- this topic is likely to take quite a bit of time
- we *will* get everyone started in programming
- IB syllabus: B2.3.1, B2.3.2, B2.3.3, B1.1.4

What is programming?

- telling the computer what to do
- traditionally, steps 3–5 in
 1. figuring out what the computer should do (requirements engineering)
 2. verifying that the computer can do it (computational complexity)
 3. deciding how the computer will do it (program design)
 4. writing the actual instructions in a programming language ("programming")
 5. converting given instructions into a form computer can use in machine instruction cycle (compilation and/or interpretation)
 6. testing that the program does what it should

Example 1

Write a Python program which

- contains two variables *a* and *b* with positive integer values
- prints the values of *a* and *b*
- prints the results of the following operations
 - $a - b$
 - division a / b
 - remainder when $3a$ is divided by $2b$

Basic properties of a Python program

- *statements* are executed sequentially
- *parameters* passed to *function calls* are in parentheses ()
- *strings* (text) are placed within single quotes " or double quotes ""
- *interpolated format strings* (f-strings)
 - are preceded by *f*
 - can contain expressions within braces {}
 - these are evaluated before output is printed
 - values of evaluated expressions are placed within string

Some Python operations

- for numerical values

operator	+	-	*	/	**	%
meaning	sum	difference	product	ratio	power	remainder

- for strings (pieces of text)

operator	+
meaning	concatenate

- for program output, function `print()`

Programming language and machine execution

- unambiguous way to instruct the computer what to do
- defined by
 - syntax (grammar): what string constitutes a valid program
 - semantics: the meaning of the string of symbols
- from programming language to machine instructions
 1. compiler (typically e.g. C)
 - programming language → executable machine instruction file
 2. interpreter (e.g. your calculator)
 - interpreter is just another program
 - interpreter reads programming language and executes it in machine code
 3. combination (e.g. Python)
 - programming language → compilation into *bytecode*
 - bytecode is simpler and faster
 - interpretation of bytecode by Python Virtual Machine (PVM)

Operations: compound $\leftrightarrow$ fundamental

- computer does simple things, *fundamental operations*
 - add, compare values, store
- we want it to do complicated things, *compound operations*
 - find interesting patterns in data
 - simulate actions in a football match
- programming transforms compound to fundamental
 1. say we want to find the sum $5 + 6 + \dots + 15$
 2. this can be achieved e.g. using two variables and a loop
 3. compiler turns this into fundamental operations
- running a program transforms fundamental to compound
 1. computer executes fundamental operations *fast*
 2. these achieve compound operation

Sum $5 + 6 + \dots + 15$ in Python and resulting bytecode * $\hookrightarrow$

- symbol * means technical details need not be learned
- symbol $\hookrightarrow$ means available at
<https://users.syk.fi/~jhurri/ib-cs/>

```
import dis # import module for showing bytecode

# function finding the sum
def fun():
    s = 0
    for i in range(5, 16):
        s += i
    return s

# disassemble the bytecode of the function
dis.dis(fun)
```

Level of abstraction

- in general, programmer strives for high level of abstraction
 - shorter code
 - easier to understand
 - fewer errors
- example: $5 + 6 + \dots + 15$ shortly
`sum(range(5, 16))`
- in fact, programmer often needs to build these abstractions for their own problem domain
- tradeoff: high level of abstraction often implies less control of details
 - example: ordering food in restaurant

Imperative programming

- state / data: variables
- statements that change state / data
- operators
- control structures

Variables and constants

- *variable* is a named storage for data in computer
- Python has *dynamic typing*

- variable can be bound to different types of data
- → type of variable is not declared
- example

```
a = 1
type(a)
a = 'this is a journey'
type(a)
```

- *constant* has fixed value during execution of program
- technically Python does not have constants
- traditionally uppercase symbols are used to indicate constants

```
NUM_MONTHS = 12
```

Some primitive data types in Python

- bool: True or False
- int: integer
- float: floating point ("decimal number")
- str: unicode string
- exploration

Operators

- constructs mapping one or multiple values into single value
 - functions with different syntax
- *multiplicity*
 - *unary*: map one value into one (e.g. $-a$)
 - *binary*: map two values into one (e.g. $a + b$)
- operators working on numbers

meaning	Python	value
assignment	=	none
augmented assignment	+=, -= etc.	none
equality	==	bool
inequality	!=	bool
less than	<	bool
less than or eq	<=	bool
integer part of quotient	//	int

- exploration

Conditionals / branching

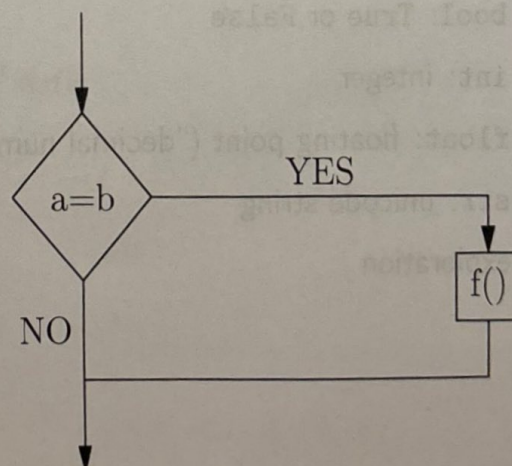
- flowchart

- arrow: sequential execution
- diamond: condition
- rectangle: process (action)

- Python

```
if a == b:  
    f()
```

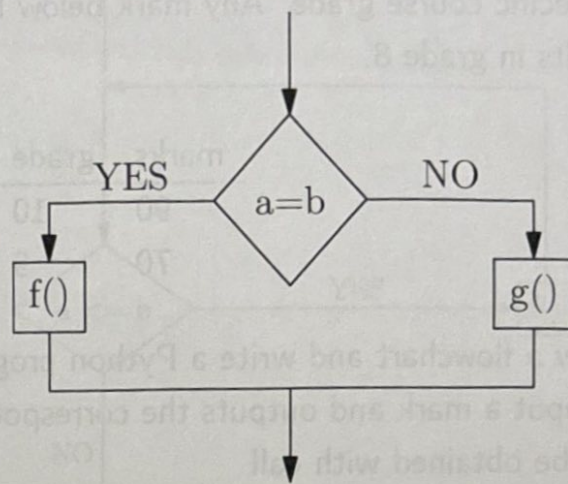
- end of effect of if is controlled by *indentation*



Alternative branches: if-else

- Python

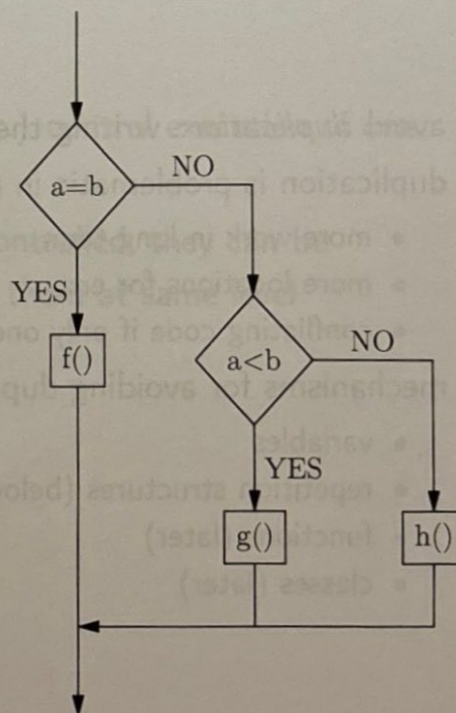
```
if a == b:  
    f()  
else:  
    g()
```



Multiple branches: if-else-if...-else

- Python

```
if a == b:  
    f()  
elif a < b:  
    g()  
else:  
    h()
```



Example 2

The following table lists the minimum number of marks required for a specific course grade. Any mark below lowest mark in the table results in grade 8.

marks	grade
90	10
70	9

Draw a flowchart and write a Python program which asks the user to input a mark and outputs the corresponding grade. Integer input can be obtained with call

```
int(input('prompt'))
```

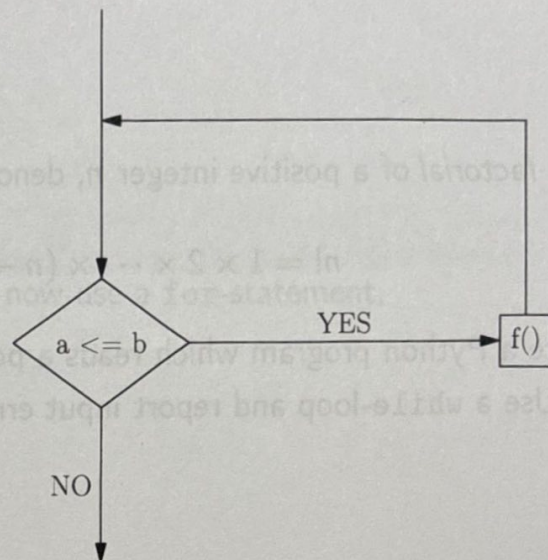
Avoid duplication

- avoid *duplication*: writing the same thing twice
- duplication is problematic in number of ways
 - more work in long term
 - more locations for errors
 - conflicting code if only one location is changed
- mechanisms for avoiding duplication
 - variables
 - repetition structures (below)
 - functions (later)
 - classes (later)

Repetition structures: while-loop

- Python

```
while a <= b:  
    f()
```



Suite ("block")

- control structures (if, while etc.) control execution of one *suite* ("block")
- if multiple statements must be controlled, they can be combined into suite by indenting them at same level

```
a = 5  
b = 7  
a_smaller = False  
  
if a < b:  
    print('a is smaller than b')  
    a_smaller = True
```

Example 3

The *factorial* of a positive integer n , denoted by $n!$, is defined by

$$n! = 1 \times 2 \times \cdots \times (n - 1) \times n.$$

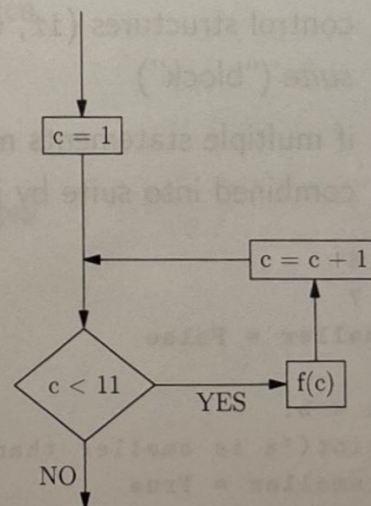
Write a Python program which reads a positive integer n outputs $n!$. Use a while-loop and report input error if n is not positive.

Repetition structures: for-statement

- Python

```
for c in range(1, 11):  
    f(c)
```

- works with any *iterable*
 - range one example
 - more on this later



Example 4

Repeat the previous task, but now use a for-statement.

Python style guide (Python Enhancement Proposal 8)

- <https://peps.python.org/pep-0008/>
- makes recommendations on programming style
- examples
 - variable: lowercase with underscores (e.g. `loading_time`)
 - function call: space only after commas (e.g. `f(2, 16)`)
 - operators
 - space around (e.g. `x = y + 5`)
 - space around lower priority (e.g. `2*x - 5`)

Example 5

Use repetition structures to write a program that outputs the given integers:

1. integers 9, 13, 17, 21, ..., 61, 65 [note: `print('text', end= '')` prints without newline]
2. 13 integers 3, 6, 12, 24, ...
3. 40 integers 1, 2, 3, -1, 5, 6, 7, -1, ..., 38, 39, -1

Example 6

Consider the following problem: find the smallest and the largest value the expression $n(n - 30)(n - 50)$ takes when n is an integer and $0 \leq n \leq 50$. Write a Python program to solve this problem.

Example 7

Consider the following problem: given a positive integer s , output the smallest value of positive integer n for which $\frac{n(n+1)}{2} > s$. Write a Python program to solve this problem. Use a while-loop and a boolean variable indicating whether the value has been found.

List of Python keywords

False	None	True	and	as
assert	async	await	break	class
continue	def	del	elif	else
except	finally	for	from	global
if	import	in	is	lambda
nonlocal	not	or	pass	raise
return	try	while		

Typical compiler steps (HL)

step	purpose
pre-processing	text manipulation (macros, comments)
lexical analysis	code to tokens (identifier, keyword)
parsing	tokens to phrases (if-then, assignment etc.)
semantic analysis	checking semantics, e.g. variable types
code generation	into machine-level code
optimisation	increase efficiency
linking	multiple code files / libs into exec

Homework 1

1. State two examples of fundamental computer operations and two examples of compound computer operations.
2. Outline the operation of a compiler.
3. Describe one reason for the use of constants.

Homework 2

Write a program which takes as input two integers a and b . The program produces the following output:

- if a and b are equal, the output is the sum of a and b squared
- otherwise the output is the sum of a and b .

Use a variable to avoid duplication.

Homework 3

Use repetition structures to write a program that outputs the given integers:

1. 10, 13, 16, ..., 34, 37
2. 998, 996, 994, ..., 900
3. 20 integers 1, -1, 1, -1, ..., 1, -1
4. 60 integers 7, 7, 9, 7, 7, 9, ..., 7, 7, 9

Homework 4

In the Blackjack card game, the dealer ("jakaja") follows the rules below to decide the next action:

total	action
less than 17	hit
17-21	stay
more than 21	bust

Write a program which reads the total and outputs the corresponding action. Avoid duplication.

Homework 5

Consider a program whose input consists of two integers a and b.

1. Write a program which outputs 1 exactly when the value of a is at least 100 and the value of b is at most 50. The output is 0 otherwise.
2. Write a program which outputs 1 exactly when the value of one of the integers is at least 100 and the value of the other one is at most 50. The output is 0 otherwise.

(Students with more programming experience would tend to use logical connectives here. Please solve this exercise using only techniques discussed so far.)

Homework 6

The price (in euros) for posting a package of weight w (in kilograms) is as follows:

weight w	price p
$w \leq 2$	3
$2 < w \leq 5$	price of posting 2 kg plus 2 euros for each additional kg
$w > 5$	price of posting 5 kg plus 3 euros for each additional kg

Let $p(w)$ be the price of posting a package weighing w .

1. Find $p(4)$. Find an expression for $p(w)$ when $2 < w \leq 5$.
2. Find $p(11)$. Find an expression for $p(w)$ when $w > 5$.
3. Write a program which reads integer w and outputs $p(w)$.

Homework 7

Consider the following problem: find the smallest value of positive integer n for which $(n^3 - 16)$ is divisible by 47. Write a Python program that solves this problem. Use a `while`-loop and a boolean variable indicating whether the value has been found. Notes: In Python, the remainder operator `%` has higher precedence (is evaluated before) addition, so remember to use parentheses.

Homework 8

1. Write a program which, given a nonnegative integer a , calculates and outputs 3^a using multiplication only.
2. Write a program which, given an integer b and a nonnegative integer a , calculates and outputs b^a using multiplication only.

Homework 9

The number of bacteria at time t (in hours) is given by

$$t(t - 20)(t - 100) + 120000,$$

where t is an integer and $0 \leq t \leq 100$. Consider the problem of finding the time at which the decrease in the number of bacteria from the previous hour is the highest. Write a Python program that solves this problem.